

A conceptual framework for agent-based agile manufacturing cells

Jianming Yu & Krishna K. Krishnan

Department of Industrial and Manufacturing Engineering, Wichita State University, Wichita, KS 67260-0035, USA, email: krishna.krishnan@wichita.edu

Abstract. *Agile manufacturing techniques are perceived as the manufacturing systems of the future. Agile manufacturing cells are dynamic and reconfigurable and the modelling of the manufacturing cells and its interaction mechanism is critical to its successful use. This paper deals with the architecture and cooperation mechanism of web-based agile manufacturing cells. Based on an analysis of structure and organization requirements of agile manufacturing cells and a comparison of three basic architectures of manufacturing systems, the quasi-heterarchical architecture is used for the agile manufacturing cell. Functional layers are defined in this architecture to make the cell's control system reconfigurable and reusable. Agent technology is adopted for implementation of each layer's functions to establish an agent-based model of agile manufacturing cells. Four types of agents including cooperation agent, job management agent, resource broker agent, and resource control agent are defined, and their functions discussed. Finally, a real time interaction mechanism of the agents is presented by considering the activities during the agents' cooperation in an agile manufacturing cell.*

Keywords: agent-based system, agile manufacturing cell, interaction mechanism, quasi-heterarchical architecture

INTRODUCTION

Agile manufacturing emphasizes high customization production, and is viewed as a breakthrough of traditional mass production (Booth, 1996). Agility is defined as the ability to thrive in an environment of continuous and unpredictable change and profit from rapidly changing, global markets for customized customer-driven products and services (Nagel & Dove, 1991). An agile corporation is adaptable, and capable of rapid reconfiguration to satisfy quickly changing market demands. Agile manufacturing crosses company borders. Core competencies of organizations can be pooled to satisfy various customer demands and reduce the time to market (Gunasekaran, 1998). One way to establish an agile manufacturing system is to employ agile manufacturing cells as building blocks to take advantage of cellular manufacturing's (CM) quick response and ability to accommodate changes in product mix (Zhang *et al.*, 1999). Cellular

manufacturing can produce medium-volume/medium-variety part types more economically than other types of manufacturing systems and surveys (Wemmerlov & John, 1997) affirm that many benefits have been gained from CM implementation including shorter manufacturing lead time and customer response time. This makes CM a logical choice for firms whose strategy is time-based competitive manufacturing. Each agile manufacturing cell in the agile manufacturing system is capable of manufacturing a part family, and is connected via the internet with other agile cells from different enterprises to seek temporary market opportunities and accomplish manufacturing tasks cooperatively. Manufacturing can take place across organizational and geographic boundaries to exploit distributed resources from different partners in dynamic teams that can be formed quickly in response to rapidly changing market demands. Internet technology is a key to enable distributed agile cells to cooperate with each other by extending traditional information systems and eliminating barriers to integration.

An agile manufacturing cell should have the ability to adapt to unpredictable changing manufacturing demands by updating its manufacturing capacity as well as functionality promptly. Efforts have been dedicated to the cell's rapid reconfiguration to enable it to accommodate fast changing market demands and high customization in a dynamic manufacturing environment. The reconfiguration activities consist of swapping existing machines between cells, adding new machines to cells, removing existing machines from cells, replacing existing machines in cells (Shewchuk, 1998), and rapid hardware changeover with modular function unit (Hollis & Quaid, 1995; Garcia *et al.*, 1996; Quinn *et al.*, 1997). However, for the manufacturing cell to work properly, the cell control system should also be reconfigurable in order to be adaptive to those cell reconfiguration activities. This paper focuses on the control system modelling of agile manufacturing cells considering their reconfigurability, responsiveness, and reusability.

RELATED WORK AND DISCUSSION

Some efforts have been made in the research of agile cell control system modelling. Chan & Zhang (2000) propose an agile manufacturing model with the Object and Knowledge-based Interval Timed Petri-Net (OKITPN) which possesses the characteristics of both object-oriented methodology and Petri-Nets. The attributes and behaviours of agile manufacturing cell can be clearly described with this model. A general Interval Timed Petri-Net (ITPN) device model is established from which specific ITPN model of different types of devices could be derived. By defining the transitions between ITPN models, the control logic of the agile manufacturing cell can be determined. The cell's quick reconfiguration and reuse are achieved by connecting the ITPN models of different devices. However, the cell control system and the manufacturing resources are tightly coupled and the control system still needs to be updated for every change in the manufacturing resources. Zhang *et al.* (1999) propose another method of modelling agile manufacturing cells also based on object-oriented methodology. Three basic types of objects are defined as the building blocks of agile manufacturing cells; they are physical manufacturing entity object (PMEO), control manufacturing entity object (CMEO), and information manufacturing entity object (IMEO). The PMEOs pertain to material flow, the IMEOs to information flow,

and the CMEO to decision-making such as routing, scheduling, etc. To accommodate the objects' interaction, each kind of object is divided into two parts: a shell for interfacing other objects and a core for processing execution. Object interoperability is guaranteed by hiding the object's non-standard natures, which is facilitated by the shell-core structure. Three functional components of the objects are defined including supervisor, functioner and executor, based on which the object interaction mechanism is derived. Kim *et al.* (1997) adopt object-oriented methodology to develop control software for agile manufacturing cell aiming to achieve rapid product changeover, rapid new product introduction, and unattended operation. The design goals are design for changes, lower complexity, and reusability. Agents are designed to implement the identified objects in the control software. Sources of workcell changes are encapsulated within classes to facilitate reconfiguration. Design patterns can be customized to solve specific design problems. A hierarchical framework is developed for error handling. However, they do not discuss the communication and cooperation issues, key issues in a distributed system. Rabelo *et al.* (1999) recognize two capabilities of an agile scheduling system: dynamic reaction to unscheduled events and global consideration of production resources beyond traditional physical boundaries. He proposes a multi-agent approach to model an agile scheduling system. Four types of generic functional agents including scheduling supervisor, enterprise activity agents, local distribution centres, and consortium, are defined based on which real scheduling systems can be customized. The consortium represents a logical and temporary clustering of distributed manufacturing resources (it is also called a virtual cell in some other researches) and is alive only if its corresponding tasks have not been completed. The information integration, communication and coordination in the community of the proposed agents are discussed. The system structure is designed to be open, and modular for easy reconfiguration.

Several issues that are essential for agile manufacturing cells still remain open. Those that have not been well addressed from the system modelling perspective by exiting research are summarized as below:

- **Reconfigurability.** Manufacturing resources may join or leave the cell at any time. The plug-and-play of devices should be realized so that the cell's manufacturing capacity and/or functionality can be enhanced to satisfy the quickly changing market demands while minimizing the downtime in its service due to these changes.
- **Responsiveness.** The agile cell should respond quickly and effectively to external events (e.g. rush orders, order changes) and internal events (e.g. machine breakdowns).
- **Reusability.** By changing the knowledge and data of the agile manufacturing cell, it can be used by different users that have different types of manufacturing resources.

Based on the analysis of cell modelling requirements in a constantly changing manufacturing environment, an architecture of agile manufacturing cell model is introduced in this paper to better address the cell's reconfigurability, responsiveness, and reusability. The key aspects of this approach are as follows:

- By comparing three different types of system architectures, the quasi-heterarchical architecture with its advantages of robustness, responsiveness and overall performance optimization is selected for modelling the agile manufacturing cell.

- Based on the function analysis of the agile manufacturing cell, four functional layers are defined in the quasi-heterarchical architecture of the agile manufacturing cell model including cooperation layer, job management layer, resource model layer and resource control layer.
- Agent technology is utilized to implement the cell model. Four types of agents are defined to implement the functions of each functional layer of the quasi-heterarchical architecture. They are cooperation agent, job management agent, resource broker agent, and resource control agent.
- The interaction mechanism of agents is modelled with Petri-net. The interaction procedure is classified into four phases: communication phase, negotiation phase, execution phase and evaluation phase. The agent interaction activities at each phase are defined.

The detailed approach is discussed in the following sections.

SYSTEM ARCHITECTURE

Architecture analysis

Agility means to be adaptable, robust, and responsive. The cell architecture is one of the key issues of its realization. Currently the most commonly used system architecture in manufacturing systems is hierarchical architecture. As shown in Figure 1(A), the node in the upper level is responsible for the job management and gives the nodes in the lower level strict schedules to follow. The nodes in the lower level perform the tasks specified in the schedules. Any variance will be sensed and processed by the upper node. The hierarchical architecture has the advantages of system stability and performance optimization, but slow response and low robustness. Because it is a fixed structure established beforehand, it lacks system extensibility and reconfigurability. Therefore, it is difficult to incorporate unforeseen changes into the system (Dilts *et al.*, 1991).

Another architecture that has received much attention is heterarchical architecture (Figure 1B) in which there is no central controller. The nodes in this structure are autonomous and selfish. They take actions independently to earn maximum profits for themselves. The relationships between nodes are both cooperative and competitive. Nodes compete for tasks and decompose them into sub-tasks by negotiating with each other. It offers the advantage of

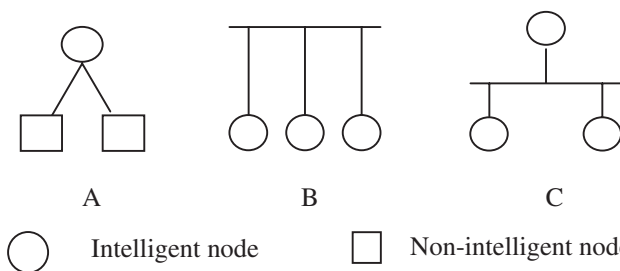


Figure 1. System architectures of manufacturing systems.

robustness (Duffie & Prabhu, 1994). Failure of nodes does not impact the system performance, because both knowledge and control algorithm are distributed. But its disadvantages are: the results are unpredictable and the global system performance optimization is very difficult to achieve (Bongaerts *et al.*, 2000).

The third architecture can be viewed as a compromise of the other two. To improve overall system performance, one or more layers can be added to the single layer heterarchical architecture. This assigns hierarchy to the system to enhance its efficiency and effectiveness on a global basis while maintaining some advantages of the heterarchical architecture. These additional layers are for the purpose of improving the intelligent nodes' global awareness. This architecture is called quasi-heterarchical architecture. As shown in Figure 1(C), the nodes in the lower level are also intelligent and have some degree of autonomy. The relationship between the upper level and lower level nodes is not a master–slave relationship, but a negotiation-based, cooperative, flexible, and dynamic one. The upper level node can delegate tasks to capable lower level nodes and control them indirectly via a certain protocol (e.g. Contract Net Protocol) to optimize the global results. The lower level node takes the responsibility of scheduling its own tasks. It can join or leave the system without influencing the system performance because nodes are loosely coupled. Also, overall system performance optimization is possible in this architecture, because the upper level node has a global view of the lower level nodes. Another important feature of this architecture is that it is quite responsive because its nodes are autonomous, which means decisions can be made locally without much time-consuming inter-node communication. Some researchers have used the quasi-heterarchical architecture for modelling agents with multiple layer architectures (Gou *et al.*, 1998). But they only applied it to the architecture of a single agent not to the architecture of a multi-agent system (MAS). This agile manufacturing cell model adopts this quasi-heterarchical architecture to satisfy the agile manufacturing cell's requirements of extensibility, reusability, responsiveness, robustness, and also the global performance optimization.

The layered structure of agile manufacturing cell

Earlier discussion reveals that quasi-heterarchical architecture is used for agile cell modelling. The next question is what the layers are. To identify the layers, a function requirement analysis of the agile manufacturing cell should be carried out first. Generally, the functions of an agile manufacturing cell can be classified into three levels: order level, job level, and operation level.

- Order level functions. As a basic unit of agile manufacturing systems, an agile manufacturing cell can utilize distributed manufacturing resources. This is realized by order delegation among cells, which requires that cells should have the capability of cooperation with each other over computer networks. Also, the cell should be able to evaluate incoming orders to determine if it can fulfill the tasks. For accepted orders, the cell functions in this level can also keep track of their status.
- Job level functions. An accepted order might contain one or more manufacturing jobs. The responsibilities of job level functions are to decompose every job into a sequence of operations,

identify the resource requirements of each operation, find the appropriate resources to do it, and monitor and report the job status.

- Operation level functions. Each job follows a sequence of operations. Each operation is delegated to a specific resource (e.g. a machine). The responsibilities of operation-level functions are to obtain tasks from the job management layer, dispatch operations to the resource by scheduling, monitor and report the resource and operation status.

Based on the function analysis, a layered structure of agile manufacturing cell is presented as shown in Figure 2. Each agile manufacturing cell has its specific functionality, i.e. the capability of manufacturing a family of similar parts, and is a functional node on the agile manufacturing network with the capability of communication with other cells distributed across shop floors or enterprises over computer networks to accomplish manufacturing tasks cooperatively. In each cell, each level of functions discussed above is represented by a functional layer in the cell model: order management layer for order level functions, job management layer for job level functions, and resource control layer for operation level functions. Note that a layer called resource model layer is inserted between the job management layer and the resource control layer, the purpose of which is to make the cell extensible and reusable. This layer maintains a cell resource model that describes the types of manufacturing capabilities of the cell. Generally, the devices in a manufacturing cell can be classified into five types: machining device, assembling device, measuring device, material handling device, and storage device. Each type of device can be classified further according to its detailed capabilities. For example, machining devices include lathe, grinding machine, drilling machine, etc. The resource model is established with all these classified types. Figure 3 is an example illustrating the structure of the model.

Each classified device type in the resource model is modelled as an intelligent node. Devices in real cells are classified accordingly and registered with corresponding nodes. The intelligent nodes can broadcast tasks assigned by the job management layer to its underlying resources. The resource model can be updated if new types of resources with new capabilities need to be

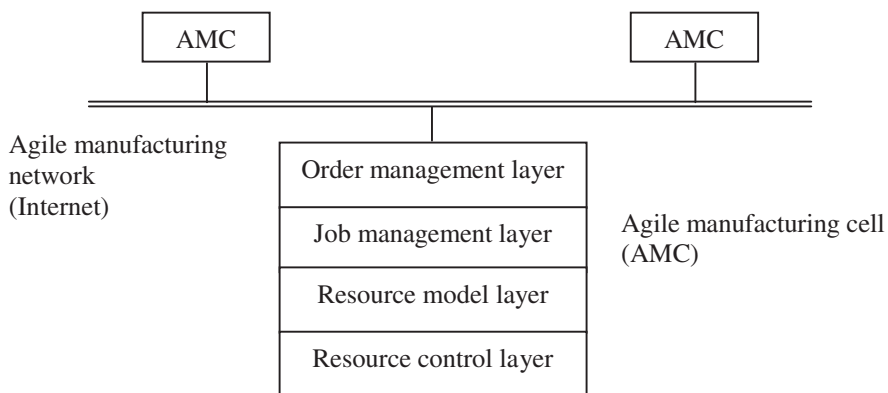


Figure 2. The layered structure of an agile manufacturing cell.

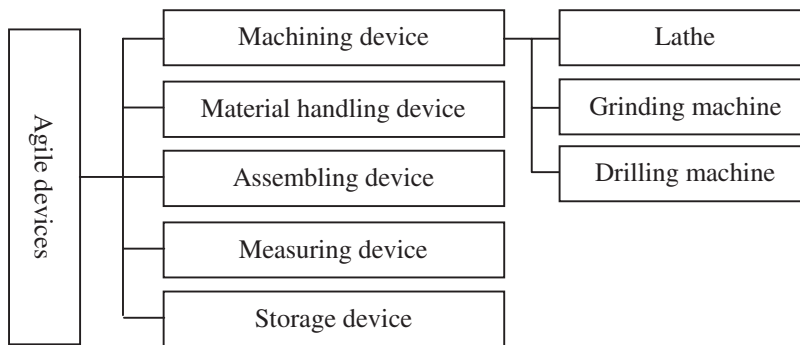


Figure 3. The agile manufacturing resource model.

added. It provides a framework for easy cell reconfiguration by separating process planning from resource allocation.

The advantage of the layered structure discussed in this section is that, with standardized interfaces between layers, changes of one layer do not influence the performance of others, which can greatly facilitate the system reconfiguration. For example, the communication protocols on the network used by the cooperation layer can be changed without impacting other layers' performance. In the resource model layer, the model can be updated to a more detailed one if the job management layer can provide more detailed resource requirements of jobs for faster dispatching. In the resource control layer, resources can join or leave it without impacting the system's functions. Moreover, given new task planning knowledge and/or new resource models, the cell can be reused in different shop floors or enterprises.

The multi-agent model of agile manufacturing cell

Agent technology has already received much attention from both academic and industrial societies. Although researchers have done much work in this area, a uniform definition of agents still does not exist. Generally, an agent is an autonomous system of software, hardware or other intelligent objects. It can make decisions and control its behaviours independently to achieve its goal based on the information it acquires from the environment via interaction. It has the following four properties (Nwana, 1996):

- **Autonomy.** The agent can perform its tasks by controlling its internal status and planning its behaviour without any interference from outside.
- **Sociability.** With communication languages, agents can interact and cooperate with other agents in an efficient way to accomplish tasks.
- **Responsiveness.** Agents can sense their environment and quickly respond to the environmental changes.

- **Proactivity.** Agents are not only reactive, but also have goal-driven behaviours. They actively acquire useful information and take appropriate actions accordingly to earn as much profits as possible.

Multi-agent system is a complex system consisting of a set of agents. Agents are independent and loosely coupled, and have both cooperative and competitive relationships. These attributes make MAS ideal to be applied to distributed system control. The MAS has the following properties (Nwana, 1996):

- **Distribution of knowledge and data.** In MAS, each agent has its own specific knowledge and local information. Agents accomplish tasks cooperatively by making local decisions and exchanging information. Data and knowledge can be shared by different agents, or can be acquired by submitting service requirements.
- **Concurrent processing.** Tasks are performed by agents independently and concurrently. Thus, the processing efficiency of the system can be greatly improved.
- **Robustness.** Agents have autonomy and loosely coupled relationships. If one of the agents cannot function well, other agents and the entire system can still work normally.
- **Extensibility.** By configuring appropriate communication protocol and control logic, as well as relevant knowledge and necessary data, a new agent can join the system without changing the system's architecture.

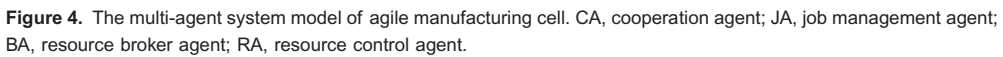
Agent technology has been applied to jobshop control and scheduling by many researchers (Sousa & Ramos, 1998; Rabelo *et al.*, 1999; Akbulut & Kamarthi, 2000; Usher & Wang, 2000). Its advantages mentioned above are compatible with the modelling requirements of agile manufacturing cells. The functions of each layer can be implemented by one or more agents. The MAS model structure of an agile manufacturing cell is shown in Figure 4. Four types of agents including cooperation agent, job management agent, resource broker agent, and resource control agent are defined for each functional layer.

Cooperation agent

A cell can cooperate with other cells over networks through the cooperation agent that has three functional modules shown in Figure 5. The cooperation and coordination module hides all the details of communication to delegate and/or acquire orders on the agile manufacturing network. It announces incoming orders to the order evaluation module. The order evaluation module is responsible for evaluating announced orders with respect to the capacities and manufacturing capabilities required, and return the evaluation results (i.e. the order is feasible or not) to the cooperation and coordination module which will try to acquire feasible orders by bidding for them. If the bidding is successful, the cooperation and coordination module will notify the order management module which will then decompose accepted orders into jobs and assign them to job management agents in the following format:

(Order ID, Job ID, Job description, Due date)

The order management module maintains the statuses of all accepted orders by referring to their corresponding job status. It will notify the cooperation and coordination module of any late



orders. The cooperation and coordination module will then try to delegate the late orders to other agents or negotiate with the customer to change the due date.

Job management agent

The job management agent has the knowledge of job analysis and resource planning. Its architecture is shown in Figure 6. It decomposes the job assigned by the cooperation agent into a sequence of operations and announces the operations to appropriate resource broker agents, respectively, according to each operation's manufacturing requirements. The announcement is as follows:

(Order ID, Job ID, Operation ID, Operation description, Starting time)

After receiving the bids of capable candidates for each operation from resource control agents, the job management agent evaluates all possible routings and delegates each operation to one candidate with the aim of minimizing overall job tardiness. The scheduling approach is not discussed in detail because it needs the system model established in this paper as its computing platform for implementation, thus the next step of this research. The number of job management agents can be dynamic, depending on factors such as the number of human experts of process planning or the number of jobs that are being processed. Job management agents can be added into or removed from the system based on needs of system scalability. The cooperation agent assigns jobs to job management agents by considering their load balancing.

Resource broker agent

The main function of the broker agent is to find capable service suppliers for customers. As shown in Figure 7, each resource broker agent represents one node of the cell's resource

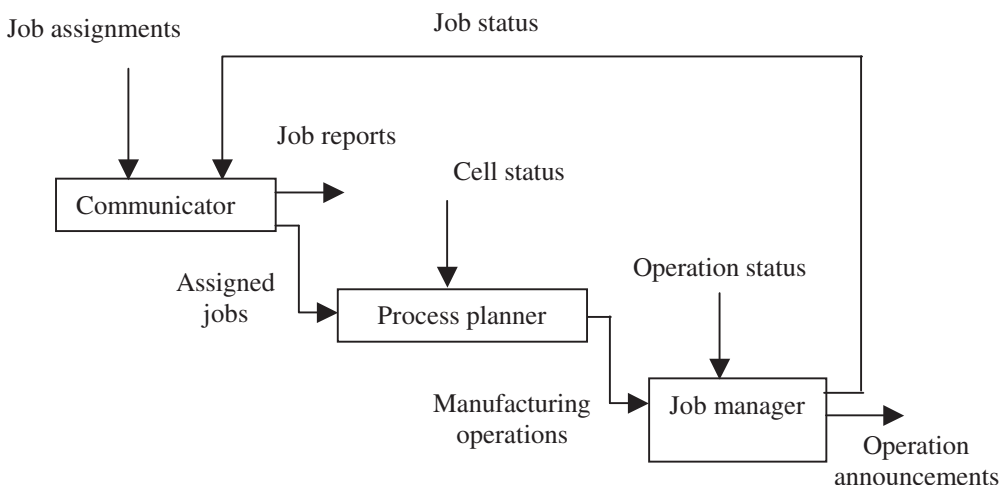


Figure 6. The IDEF0 graph of the job management agent.

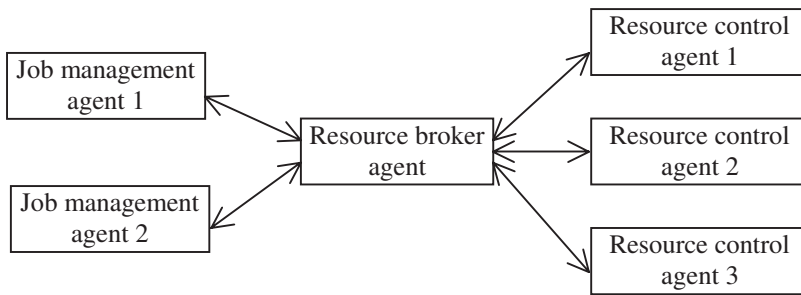


Figure 7. The brokering model.

model. The resource broker agent is responsible for the management of the underlying resources (represented by resource control agents) involving resource registration or cancellation. Resources can be from different cells or even different shop floors. The resource broker agent broadcasts incoming tasks with respect to their manufacturing requirements to its underlying resources, and sends back the bids from all candidates to the corresponding job management agent. The manufacturing resources of agile manufacturing cells tend to be reconfigured frequently to satisfy constantly changing demands. In the brokering model, job management agents do not need to memorize the information regarding all available resources and update it along with the changes of the resources, which are the responsibilities of the resource broker agent. This makes agile cells much easier to be reconfigured by separating task planning from resource allocation. Also, the brokering model facilitates distributed resources utilization, which is one of the key points of agile manufacturing.

Resource control agent

The resource control agent is responsible for the scheduling/rescheduling, monitoring and status reporting of its resource, namely the device. Resource control agents can be from distributed sites. They register with one or more of the resource broker agents to bid for operations. After receiving the information of the operation broadcasted by resource broker agents, the resource control agent evaluates its machine's status and capacity to judge if the machine can do the operation. If it can, the agent will find a time slot for this operation by considering all the operations it holds, and submit a bid that has the following format:

(Order ID, Job ID, operation ID, earliest beginning time, latest beginning time, estimated finishing time)

The resource agent's working procedure is shown in Figure 8. If the bid is accepted by the job management agent, the resource control agent will dispatch the operation to its device.

Summarizing the functions of the four types of agents discussed in the section 'The multi-agent model of agile manufacturing cell', Figure 9 gives a global view of the cooperation mechanism among them. Orders are acquired from the agile manufacturing network, and then trans-

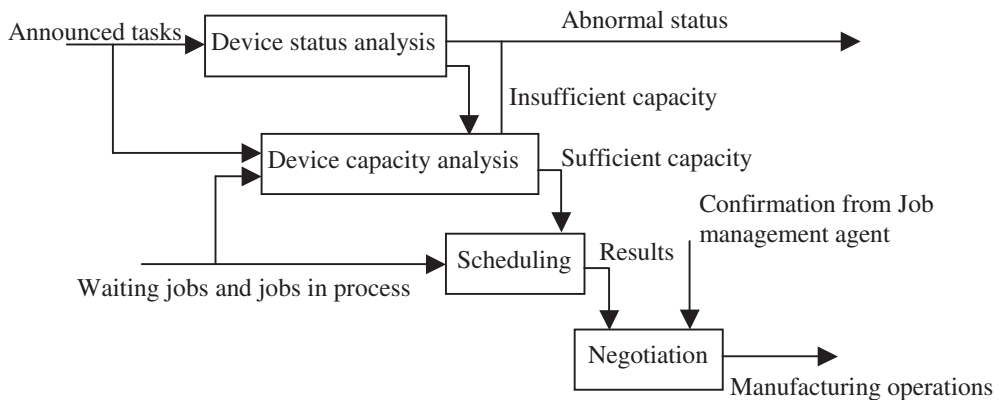


Figure 8. The resource control agent's working procedure.

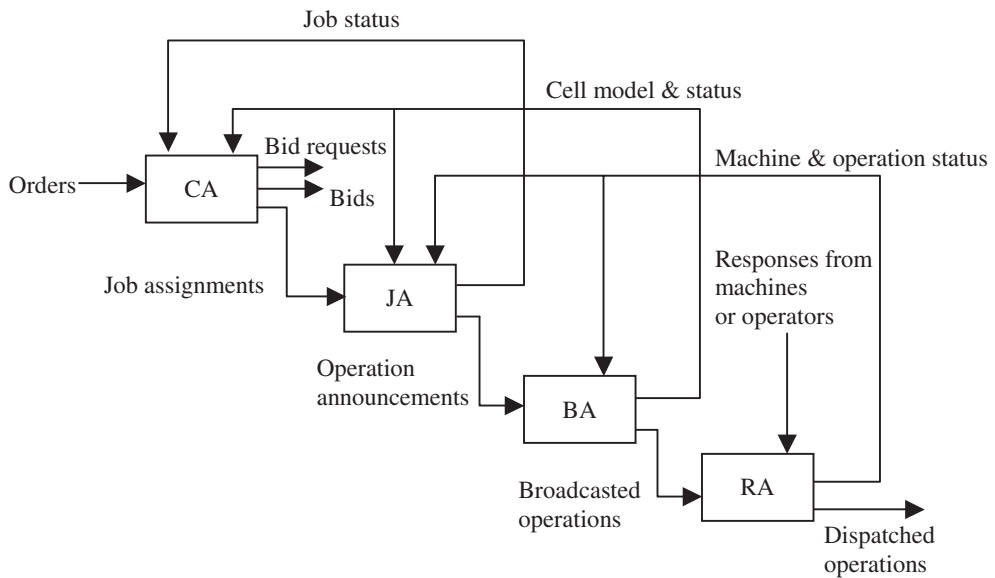


Figure 9. The IDEF0 graph of the cooperation among agents.

ferred downwards in the system as jobs and operations. Various levels of system information including machine and operation information, job status and cell status are transferred upwards to corresponding agents who need it for decision-making or prompt reaction. During the cooperation, various agent activities may occur which may lead to unpredictable system results. In order for the agents to cooperate more efficiently, in the next section, we propose an agent interaction activity model that can reduce unnecessary interagent communication and facilitate the system implementation.

INTERACTION MECHANISM

Agent interaction is a very important aspect in multi-agent-based systems because it determines the cooperation efficiency and effectiveness. In the proposed cell model, the interaction activities of agents are driven by outside bid requests or inside order/job/operation-related events (e.g. rush orders, order changes, machine breakdowns), which may happen at any time during the cell's service. A real time interaction mechanism should be built into the system to let the information of those events or disturbances to be delivered to the agents who are responsible for handling them and to those that are affected by them so that these agents can take actions promptly. The responsiveness of the whole system is improved in this way. Given the quasi-heterarchical system architecture, it is convenient to model the agent interaction with a top-down method. From a global point of view, the interaction activities are classified into four phases: communication phase, negotiation phase, execution phase, and evaluation phase. In the communication phase, the agent at the upper layer tries to contact appropriate agents of the lower layer. Information such as job assignments, operation broadcasts are transmitted across layers; in the negotiation phase, the job management agent negotiates with resource control agents that are found by the broker agent, and makes a decision to delegate the operation to one of them; in the execution phase, the operations are being processed by the resource control agent. Any changes to the operation may lead the interaction back to the previous two phases. If the operation is finished on time, the interaction gets to the final evaluation phase in which the operation results and the resource control agent's performance is evaluated by the job management agent.

Each phase of the interaction contains several agent behaviours that are prerequisites of the interaction or the results of it. To visualize the interaction mechanism, the interaction activity model of the agents (cooperation agent, job management agent, resource broker agent, and resource control agent), for accomplishing manufacturing jobs cooperatively is modelled on Petri-Net as shown in Figure 10. The model gives a complete description of the interaction procedure by considering all possible interaction activities of the agents' cooperation. But it does not impact the agents' autonomies by interfering with the agents' decision-making procedures. It only specifies the necessary requests and responses as each agent cooperates with other agents. This facilitates the system integration in heterogeneous industrial environments, and keeps the interagent communication lean.

In Figure 10, $T1_a$, $T2_b$, $T3_c$, and $T4$ are transitions that are triggered by agents' corresponding behaviours in the communication phase, the negotiation phase, the execution phase, and the evaluation phase of the interaction, respectively. Interaction relationships are dynamically established. Possible agent behaviours at the four phases are all modelled as transition conditions. The places, $P1_i$, $P2_j$, $P3_k$, and $P4$, where decisions may have to be made by agents, represent system situations of the four phases.

The description of the places are listed below:

$P1_0$: CA assigns jobs to JA by considering load balancing.

$P1_1$: JA generates an operation list and contacts corresponding BA for each operation.

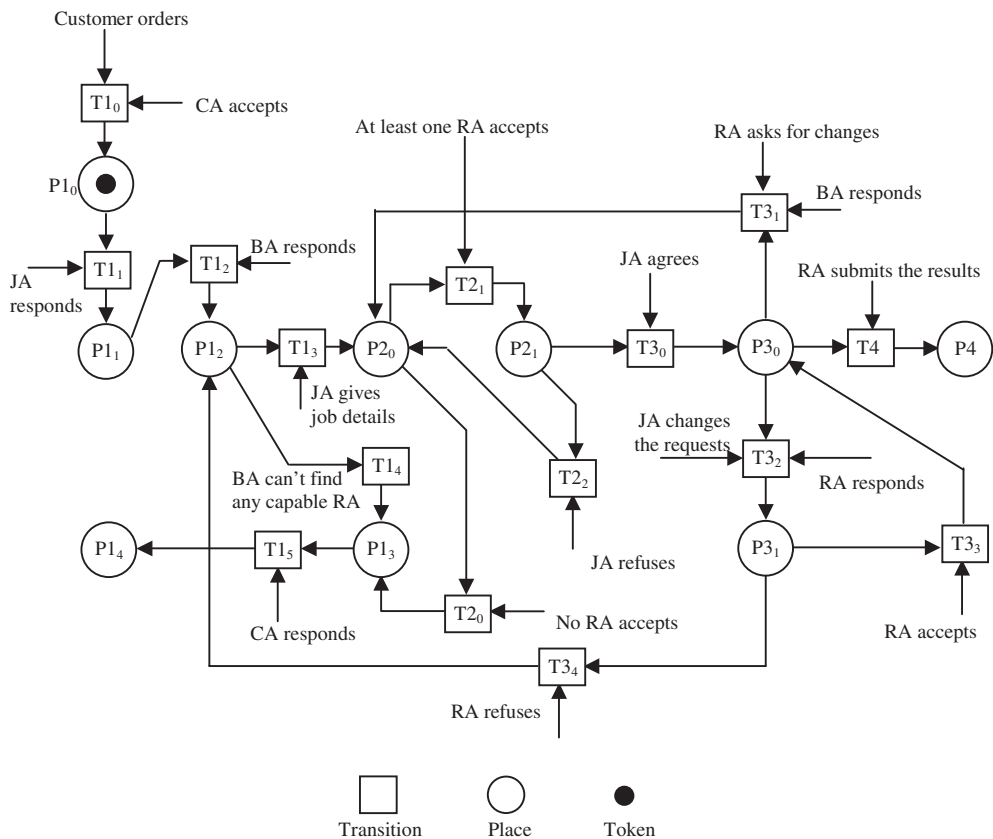


Figure 10. The agent interaction model.

P1₂: BA tries to find appropriate RAs registered with it to broadcast the operations.

P1₃: BA cannot find any appropriate RA. It notifies JA which then notifies CA.

P1₄: CA tries to delegate the jobs to other cells or negotiate with the customer about the due dates.

P2₀: Operations are announced to RAs.

P2₁: RAs return their bids to the JA who will then make a choice.

P3₀: Operations have been released to the selected RA and are in progress.

P3₁: JA gives the detailed changes and negotiates with the related RA.

P₄: Finished operations have been submitted to the JA who will give awards or penalties to the corresponding RA by comparing the original operation requirements with the actual results.

The interaction between agents is message-based. The basic interaction operations are *send* and *receive*. Each agent is equipped with a messenger for interpreting incoming mes-

sages as well as formatting outgoing messages, a mailbox for temporarily storing the message queue, and a directory that will be dynamically updated according to current transactions. With the agent interaction activity model, the requirements and responsibilities of each agent are clearly defined which facilitates the agent's implementation. The detailed interaction protocol can then be derived based on the defined activities and their corresponding participants. Because this work focuses on the system modelling methodology, the detailed protocol is not discussed.

SYSTEM IMPLEMENTATION AND FEATURES

A prototype of the proposed agile manufacturing cell is being developed in the automation lab at Wichita State University. Visual Basic with DCOM technology is used to implement the cell's agents and architecture. Active Server pages (ASP) based on Internet Information Server (IIS) on a Windows 2000 platform is used to develop the interface to each cell to make it accessible on the internet. In addition to the advantages of its layered structure, there are some system performance related features of the prototype:

- The system has an organization structure and a built-in interaction mechanism, which enable it to respond quickly to disturbances. For example, if one of the resource control agents finds that its resource entity has broken down and some of the operations cannot be accomplished, it will contact the resource broker agent to delegate the affected operations to other resource agents. The resource broker agent will broadcast these operations to all capable resources registered with it and return their bids to the resource control agent who will then judge to which agent it will delegate its operations. This procedure greatly reduces the communication load and time because the resource model provides a clear view of the system and the resource broker agent knows whom it should broadcast the affected operations. If the resource broker agent finds that there is no capable resource agent available for those operations, it reports this to the job management agent who will then notify the cooperation agent. The cooperation agent will delegate these jobs to other cells or negotiate with the customer who placed the order to change the due date.
- Multiple job management agents improve the cell's efficiency by facilitating parallel processing.
- Convenience for tracking job status, or changing the jobs' due dates or cancelling them according to customer's demands. Via job management agents and the interaction mechanism, corresponding resource control agents can be notified of these changes promptly and update their schedules accordingly in an agile way.
- Manufacturing resources of exiting types that are already described in the resource model can be added into or removed from cells in real time without interrupting the cell's services. These cell reconfiguration activities are facilitated by resource broker agents, which can provide on line services of resource registration or cancellation.

CONCLUSIONS

This paper described the first step in the goal to develop a responsive, robust, and reconfigurable agile manufacturing system. Agent-based agile manufacturing cells are used as building blocks of such an agile manufacturing system to exploit the benefits of cellular manufacturing and agent technology. For this purpose, an agent-based model of agile manufacturing cells is presented, which has a layered structure and consists of a set of intelligent agents for intercell cooperation, job management, and resource control. With the aim to adapt to quickly changing market demands, the layered structure of the proposed model enables the cell to be reconfigurable and reusable. However, to improve the performance of agile manufacturing cells, the model also needs to facilitate parallel job processing, efficient communication and coordination, and quick system response in a dynamic manufacturing environment. These goals are achieved by employing different types of intelligent agents to implement the four functional layers of the cell model with the agents' functions and their interaction mechanism clearly defined.

The proposed agent-based cell model provides a computing platform for the implementation of distributed scheduling techniques for agile manufacturing. A robust and high performance scheduling approach for efficiently utilizing the distributed resources of agile manufacturing cells will be the focus of the future study.

REFERENCES

- Akbulut, M.B. & Kamarthi, S. (2000) An agent-based scheduling system to achieve agility. *Proceedings of SPIE – the International Society for Optical Engineering*, **4208**, 19–30.
- Bongaerts, L., Monostori, L., McFarlane, D. & Kadar, B. (2000) Hierarchical in distributed shop floor control. *Computers in Industry*, **43**, 124–137.
- Booth, R. (1996) Agile manufacturing. *Journal of Engineering Management*, **April**, 105–112.
- Chan, T.S. & Zhang, J. (2000) Modeling for agile manufacturing systems. *International Journal of Production Research*, **39**, 2313–2332.
- Dilts, D.M., Boyd, N.P. & Whorms, H.H. (1991) The evolution of control architectures for automated manufacturing systems. *Journal of Manufacturing Systems*, **10**, 102–115.
- Duffie, N.A. & Prabhu, V.V. (1994) Real time distributed scheduling of heterarchical manufacturing. *Journal of Manufacturing Systems*, **13**, 94–107.
- Garcia, P., Woods, R., Rebeil, J., Jones, J. & Griesmeyer, J. (1996) Agile manufacturing prototype system. *Proceedings of the 5th Agile Manufacturing Enterprise Forum Conference*. Agile Manufacturing Enterprise Forum, Boston, MA, pp. 395–416.
- Gou, L., Luh, P.B. & Kyoya, Y. (1998) Holonic manufacturing scheduling: architecture, cooperation mechanism, and implementation. *Computers in Industry*, **37**, 213–231.
- Gunasekaran, A. (1998) Agile manufacturing enablers and an implementation framework. *International Journal of Production Research*, **36**, 1223–1247.
- Hollis, R. & Quaid, A. (1995) An architecture for agile assembly. *Proceedings of American Society of Precision Engineering, 10th Annual Meeting*. Austin, TX, pp. 211–225.
- Kim, Y., Jo, J., Velasco, V.B., Barendt, N.A., Podgurski, A., Ozsoyoglu, G. et al. (1997) A flexible software architecture for agile manufacturing. *Proceedings of IEEE International Conference on Robotics and Automation*, April, Albuquerque, NM, **4**, 3043–3047.
- Nagel, R. & Dove, R. (1991) *21st Century Manufacturing Enterprise Strategy*. Iacocca Institute. Lehigh University, Pennsylvania, USA.
- Nwana, H.S. (1996) Software agent: an overview. *Knowledge Engineering Review*, **11**, 205–244.
- Quinn, R., Causey, G.C., Merat, F.L., Sargent, D.M., Barendt, N.A., Newman, W. et al. (1997) An agile

manufacturing workcell design. *IIE Transactions*, **29**, 901–909.

Rabelo, R.J., Camarinha-Matos, L.M. & Afsarmanesh, H. (1999) Multi-agent-based agile scheduling. *Robotics and Autonomous Systems*, **27**, 15–28.

Shewchuk, J. (1998) A framework for modeling and design of agile discrete-part manufacturing facilities. *Proceedings of the 7th Industrial Engineering Research Conference*. Alberta, Canada, pp. 198–216.

Sousa, P. & Ramos, C. (1998) A dynamic scheduling holon for manufacturing orders. *Journal of Intelligent Manufacturing*, **9**, 107–112.

Usher, J.M. & Wang, Y.-C. (2000) Intelligent agents architectures for manufacturing control. *Proceedings of 9th Industrial Engineering Research Conference*, Cleveland, OH, May 21–23, (CD-ROM).

Wemmerlov, U. & John, D. (1997) Cellular manufacturing at 46 user plants: implementation experiences and performance improvements. *International Journal of Production Research*, **35**, 29–49.

Zhang, J., Gu, J., Li, P. & Duan, Z. (1999) Object-oriented modeling of control system for agile manufacturing cells. *International Journal of Production Economics*, **62**, 145–153.

Biographies

Jianming Yu is currently a PhD student at the Department of Industrial and Manufacturing Engineering of Wichita State University, KS, USA. He obtained his bachelor's degree in 1992 and master's degree in 1997 in Mechanical Engineering both from Shandong University in China. His research interests include agile manufacturing systems, agent-based jobshop scheduling, CAD, assembly modelling, internet technology and its application in manufacturing industry.

Krishna K. Krishnan is an associate professor of Industrial and Manufacturing Engineering at Wichita State University. He graduated from Virginia Tech in 1994 with a PhD in Industrial and Systems Engineering. He has a BS degree in Mechanical engineering and an MS in Industrial and Systems Engineering. Dr Krishnan teaches courses on Web-Based Automation, Computer Integrated Manufacturing and Computer-Aided Design and Manufacturing. Dr Krishnan's primary research interests are in Integrated Manufacturing Systems, CAD/CAM, and Free-Form Surface Manufacture.